

Waraqah

Project Planning Document

Farhan Shahriyar Hossain, Rahinur Bin Naushad,
Al Mukit Muktafin, Arifin Rafi

A Flutter-Based Book Marketplace, Peer-to-Peer Resale,
Social Reading Feed, AI Recommendations, and Islamic Curation

Frontend:	Flutter / Dart
Architecture:	Feature-Based LEGO Architecture
Backend:	Supabase (Postgres, Auth, Storage, Edge Functions)
Database:	PostgreSQL (managed by Supabase)
AI:	Google Gemini API (proxied via Supabase Edge Function)
Navigation:	GoRouter
Animation:	Rive
Image Storage:	Cloudinary

September 20, 2026

Contents

1	Project Overview	2
2	Technology Stack	2
3	Architecture	2
4	Project Structure	3
5	Backend Architecture (Supabase)	4
6	Database	5
7	Main Features	6
7.1	Primary Marketplace	6
7.2	Second-Hand Marketplace (P2P)	6
7.3	Book-Bites (Social Feed)	6
7.4	AI Reading Assistant	6
7.5	Islamic Curation	6
8	Navigation	6
9	Data Flow	7
10	Development Phases	7
10.1	Phase 1 – Project Foundation	8
10.2	Phase 2 – Authentication	8
10.3	Phase 3 – Primary Marketplace	8
10.4	Phase 4 – Second-Hand Marketplace	8
10.5	Phase 5 – Book-Bites	8
10.6	Phase 6 – AI Assistant	8
10.7	Phase 7 – Islamic Features	8
10.8	Phase 8 – Cart & Checkout	9
10.9	Phase 9 – Testing	9
11	Security	9
12	Final Architecture	9

1 Project Overview

Waraqah is a mobile application that helps users buy new books, resell used books, and stay engaged with reading through a social feed and an AI assistant.

The application will combine a primary book marketplace, a peer-to-peer resale marketplace, a social reading feed, an AI recommendation assistant, and Islamic-oriented content curation.

The main features are:

- Primary marketplace for new books, with cross-vendor price comparison.
- Peer-to-peer marketplace for second-hand books (students reselling textbooks).
- “Book-Bites” social feed with inline book tagging.
- AI reading assistant powered by Gemini.
- Beneficial / non-beneficial book categorization.
- Daily Ayah of the Qur’an on the home screen.
- Cart and checkout (mocked payment for this project).

2 Technology Stack

Technology	Purpose
Flutter	Mobile application development
Dart	Flutter programming language
Supabase	Backend-as-a-Service: Postgres, Auth, Storage, RLS, Edge Functions
PostgreSQL	Main application database (managed by Supabase)
GoRouter	Application navigation
Gemini API	AI-powered reading recommendations
Cloudinary	Book cover and listing image storage
Rive	Selected animations and interactive feedback
supabase_flutter SDK	Flutter–Supabase communication

3 Architecture

The Flutter application will follow a **feature-based LEGO architecture**.

Each major feature will act as an independent building block. A feature will contain its own UI, models, data handling, and business-related components.

This makes the application easier to:

- Develop.
- Test.
- Maintain.
- Debug.
- Extend.
- Replace individual features.

Shared functionality (the **Book** model, cross-feature interfaces, theming) will remain inside **core/**.

Within each feature, internal code will follow Clean Architecture, separated into **presentation**, **domain**, and **data** layers.

The backend has moved from a custom Go REST API to **Supabase**. Supabase provides the PostgreSQL database, authentication, and file storage directly, so a feature's **data** layer talks to the Supabase client SDK instead of a hand-written REST endpoint. Authorization that would previously have lived in Go now lives in the database as Row Level Security (RLS) policies. The one exception is the Gemini API call, which is kept behind a small Supabase Edge Function so the API key is never shipped inside the app.

4 Project Structure

The basic Flutter structure will be:

```
lib/  
|  
+-- main.dart  
|  
+-- app/  
|   +-- app.dart  
|   +-- router/  
|       +-- app_router.dart  
|  
+-- core/  
|   +-- network/  
|   +-- storage/  
|   +-- theme/  
|   +-- widgets/  
|   +-- services/  
|   +-- utils/  
|  
+-- features/  
    |  
    +-- auth/  
    +-- home/  
    +-- catalog/  
    +-- p2p/  
    +-- book_bites/  
    +-- ai_assistant/  
    +-- cart/  
    +-- checkout/  
    +-- profile/
```

Each feature may follow:

```
feature/  
|  
+-- presentation/  
+-- domain/  
+-- data/
```

5 Backend Architecture (Supabase)

The backend is provided by **Supabase**, built on PostgreSQL.

Flutter communicates with Supabase directly through the official `supabase_flutter` SDK. Users are authenticated with Supabase Auth, which issues a session tied to `auth.users`; a matching row in a `profiles` table is created automatically on signup via a database trigger (`handle_new_user`).

Supabase is responsible for:

- Authentication and session management.
- User profiles.
- Book catalog data.
- P2P listings.
- Social feed (Book-Bites), including tags and likes.
- Cart and checkout (mocked orders).
- Reviews.
- Ayah of the Day content.

Authorization is no longer enforced by application-layer backend code. Instead, every table has **Row Level Security (RLS)** enabled, with policies that decide who may `select`, `insert`, `update`, or `delete` each row (for example, a user may only edit their own P2P listing, cart, or order, checked against `auth.uid()`).

Because the Gemini API key must never be embedded in the Flutter app, AI requests are proxied through a single **Supabase Edge Function**, which holds the key server-side and injects catalog context into the prompt before calling Gemini.

Vendor scraping (Rokomari, Wafilife) and cross-vendor price comparison are not yet represented in the Supabase schema — the current `books` table stores a single price per title. This remains a planned extension of the schema (see Section 10.3).

The general architecture is:

```

Flutter App
  |
  | supabase_flutter SDK
  v
Supabase
  |
  +-----> PostgreSQL (RLS-protected)
  |
  +-----> Supabase Auth
  |
  +-----> Edge Function ---> Gemini API
  |
  +-----> Cloudinary (image storage)

```

The Flutter application never opens a raw connection to PostgreSQL; every request goes through Supabase’s client libraries and is governed by RLS rather than by a separate API layer.

6 Database

PostgreSQL, managed by **Supabase**, is used as the main database. The schema currently contains the following tables.

- **profiles** — extends Supabase’s `auth.users` with app fields (`full_name`, `university`, `student_id`, `avatar_url`); a row is created automatically on signup via the `on_auth_user_created` trigger.
- **books** — cached from the Google Books API (Open Library fallback), keyed by `google.books_id` / `open_library_id`; stores `title`, `author`, `isbn`, `genre`, `cover_url`, a single `price/currency`, and an `is_beneficial` flag for Islamic curation (indexed on `title` and `is_beneficial`).
- **p2p_listings** — second-hand listings, optionally linked to a `books` row (`book_id`, nullable); stores `seller_id`, `condition` (`new/like_new/good/fair/poor`), `price`, `photo_urls` (Cloudinary URLs), and `status` (`available/reserved/sold`).
- **posts** — Book-Bites posts, with `author_id` and `content`, indexed on `created_at desc` for feed pagination.
- **post_book_tags** — join table linking a post to the books it tags (`post_id`, `book_id`), unique per pair.
- **post_likes** — one like per user per post (`post_id`, `user_id`, unique together).
- **cart_items** — one row per user/book with a `quantity`; re-adding the same book bumps the quantity instead of duplicating the row.
- **orders** and **order_items** — mocked checkout: an order has a `status` (`confirmed/cancelled`) and `total_amount`; each order item snapshots `price_at_purchase` so later price changes don’t affect past orders.
- **reviews** — one review per user per book, with a `rating` (1–5) and optional `comment`.

PostgreSQL is suitable for this project because the marketplace, P2P listings, and social feed data are all relational, with clear foreign-key relationships between users, books, and posts.

Row Level Security is enabled on every table. Public data — profiles, books, P2P listings, posts, tags, likes, and reviews — is readable by anyone (for `select using (true)`), with writes restricted to the owning row’s user, checked against `auth.uid()` (e.g. `auth.uid() = seller_id` for

a P2P listing, `auth.uid() = author_id` for a post). Books currently have no public insert/update policy, so catalog data is written server-side (dashboard or a trusted admin role). Private data — cart items, orders, and order items — is restricted to `select/insert/update/delete` by its owner only; order items check ownership through the parent order's `user_id`.

7 Main Features

7.1 Primary Marketplace

Users can browse and buy new books.

- Physical books and ebooks.
- Price comparison across vendors for the same title/edition.
- Default sort: cheapest first, ties broken by average review score.
- Reviews, previews, and ratings shown per book.

7.2 Second-Hand Marketplace (P2P)

Users can resell books they no longer need.

- Aimed at students reselling course textbooks.
- Separate listing flow: condition reporting, pricing, photos.
- Independent from the primary catalog.

7.3 Book-Bites (Social Feed)

A lightweight, Twitter-style feed for readers.

- Short posts, reading progress, quick reviews.
- Inline book tagging: tap a tagged book to open its purchase page.
- Paginated feed backed by PostgreSQL.

7.4 AI Reading Assistant

An embedded chatbot for personalized suggestions.

- Powered by the free Google Gemini API.
- A Supabase Edge Function proxies all requests; the app never holds the API key.
- Catalog data is injected into the prompt so recommendations stay within Waraqah's own listings.

7.5 Islamic Curation

- Books are tagged *Beneficial* or *Non-Beneficial* and can be filtered accordingly.
- A daily Ayah of the Qur'an is shown at the top of the home screen.

8 Navigation

GoRouter will be used for application navigation.

Main routes will include:

```

/login
/register
/home
/catalog
/catalog/book/:id
/p2p
/p2p/:id
/p2p/create
/book-bites
/book-bites/create
/ai-chat
/cart
/checkout
/orders
/profile

```

Authentication-protected routes will be handled by route guards, driven by the current Supabase Auth session state.

9 Data Flow

The general application flow will be:

```

Flutter UI
  |
  v
Feature Controller
  |
  v
Repository
  |
  v
Supabase Client SDK
  |
  +-----> PostgreSQL (RLS)
  |
  +-----> Supabase Auth
  |
  +-----> Edge Function -> Gemini API
  |
  +-----> Cloudinary

```

This keeps the UI separate from Supabase's implementation. There is no custom middle API: access control is enforced at the data layer by RLS instead of by backend business logic.

10 Development Phases

10.1 Phase 1 – Project Foundation

- Create Flutter project.
- Set up LEGO architecture.
- Configure GoRouter.
- Create Supabase project and apply the initial schema.
- Add `supabase_flutter` to the app and configure the client.

10.2 Phase 2 – Authentication

- Registration.
- Login.
- Session handling via Supabase Auth.
- Profile (backed by the `profiles` table and its signup trigger).

10.3 Phase 3 – Primary Marketplace

- Extend the `books` schema for multi-vendor pricing (currently a single price per title).
- Vendor scraper workers (Rokomari, Wafilife), run outside the Flutter app and written back to Supabase.
- Entity resolution / deduplication across vendors.
- Catalog browsing and search.
- Price comparison and sorting.

10.4 Phase 4 – Second-Hand Marketplace

- Listing creation.
- Condition reporting.
- P2P browsing and search.

10.5 Phase 5 – Book-Bites

- Post creation.
- Feed pagination.
- Inline book tagging.

10.6 Phase 6 – AI Assistant

- Gemini API integration via a Supabase Edge Function.
- Context injection (catalog-aware prompts).
- Chat UI.

10.7 Phase 7 – Islamic Features

- Beneficial / non-beneficial categorization.
- Ayah of the Day on the home screen.

10.8 Phase 8 – Cart & Checkout

- Cart.
- Mocked checkout flow (no real payment gateway).
- Order confirmation.

10.9 Phase 9 – Testing

- Unit testing.
- Widget testing.
- RLS policy testing.
- Final UI and performance improvements.

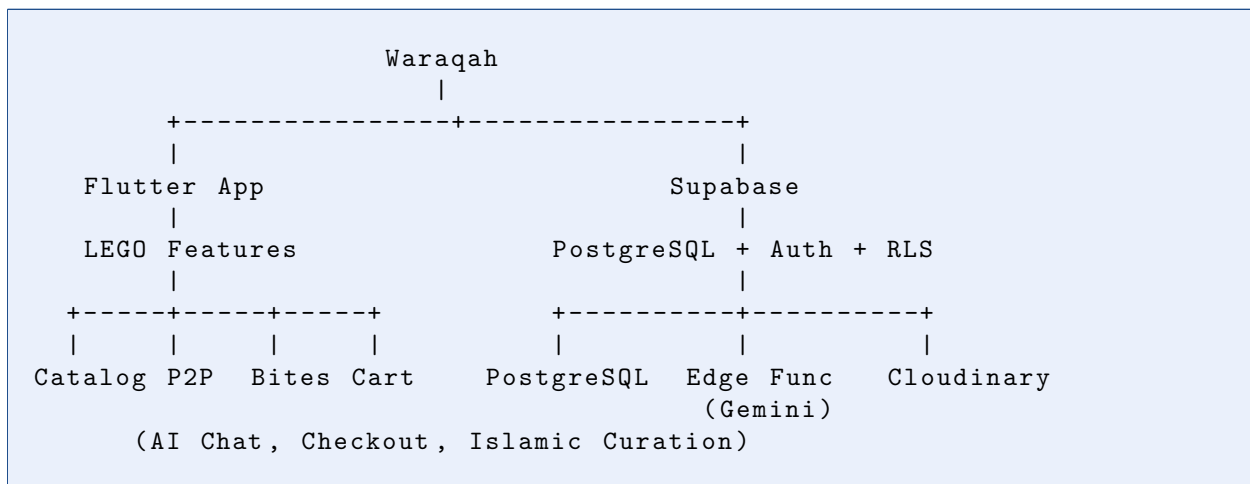
11 Security

Important security rules:

- Flutter connects to PostgreSQL only through the Supabase client SDK, governed by Row Level Security — never with a raw database connection.
- The Gemini API key is never embedded in the Flutter app; it is held server-side by the Supabase Edge Function that proxies AI requests.
- Authentication is handled by Supabase Auth.
- Row Level Security policies validate ownership on every insert, update, and delete (e.g. a user can only edit their own P2P listing, cart, or order).
- Checkout is mocked for this project; no real payment credentials are handled.

12 Final Architecture

The complete system can be summarized as:



The main goal is to keep Waraqah modular and simple.

Flutter will handle the mobile application, and Supabase will provide the database, authentication, storage, and access control (via RLS) in place of a custom backend. A single Edge Function stands in for the AI-proxy responsibility the Go backend previously held. Each major feature will remain

an independent LEGO-style module, communicating with Supabase directly through the client SDK.